

Guía de actividades: RECORD sobre BANK SOLUTIONS

Sesión 1.1.1 · Indicador de logro 1.1 · 4 ejercicios · Documento del estudiante

Asignatura	BDY1103 – Taller de Base de Datos	Experiencia	EA1
Tiempo	50 min en clase + trabajo autónomo	Modalidad	Individual
Herramienta	Oracle SQL Developer	Base de datos	BANK SOLUTIONS

Contexto de negocio

BANK SOLUTIONS es una institución financiera que administra productos de inversión y créditos para sus clientes. Su área de sistemas necesita rediseñar varios procesos internos para generar información de forma más eficiente: fichas de clientes, perfiles comerciales y estados de crédito.

En esta actividad resolverás cuatro requerimientos de información utilizando bloques PL/SQL anónimos y estructuras de tipo RECORD. El primero está resuelto y lo desarrollará el docente en clase; los otros tres los resuelves tú.

Preparación del ambiente

1. Conéctate en SQL Developer con tu usuario de trabajo.
2. Si aún no lo has hecho, ejecuta el script crea_pobla_tablas_bd_BANK_SOLUTIONS.sql.
3. Verifica la carga con la siguiente consulta. Debe devolver 130.

```
SELECT COUNT(*) FROM CLIENTE;
```

4. Activa la salida por pantalla antes de ejecutar cualquier bloque:

```
SET SERVEROUTPUT ON;
```

Entrega: guarda cada ejercicio en un archivo .sql independiente, con el nombre EJ2_apellido.sql, EJ3_apellido.sql y EJ4_apellido.sql, y súbelos al AVA al final de la sesión. Incluye en cada archivo una captura o el texto de la salida obtenida.

Ejercicio 1 — Ficha de cliente

Requerimiento

El área de atención al cliente necesita imprimir la ficha básica de un cliente a partir de su número interno: RUN con dígito verificador, nombre completo, dirección y fecha de inscripción. Construye un bloque PL/SQL anónimo que muestre la ficha del cliente número 3.

Análisis previo

Los cuatro datos que necesitamos son de distinto tipo y describen a una misma persona, así que corresponden a un RECORD. No usaremos %ROWTYPE porque no queremos las dieciocho columnas de CLIENTE: queremos cuatro, y dos de ellas son valores construidos (el RUN concatenado con su dígito verificador y el nombre completo armado a partir de cuatro columnas).

Solución

```
SET SERVEROUTPUT ON;

DECLARE
-- PASO 1: definimos el molde. Esto todavía no crea ninguna variable.
TYPE t_ficha_cliente IS RECORD (
    run          VARCHAR2(15),
    nombre       VARCHAR2(120),
    direccion    CLIENTE.direccion%TYPE,
    inscripcion  CLIENTE.fecha_inscripcion%TYPE
);

-- PASO 2: creamos la variable. Nace con sus campos en NULL.
v_ficha  t_ficha_cliente;
v_nro    CLIENTE.nro_cliente%TYPE := 3;
BEGIN
-- PASO 3: cargamos el RECORD completo con un unico SELECT ... INTO.
-- El orden de las columnas debe calzar con el orden de los campos.
SELECT TO_CHAR(numrun) || '-' || dvrn,
       TRIM(pnombre) || ' ' || NVL(snobre,'') || ' ' ||
       apaterno || ' ' || NVL(apaterno,''),
       direccion,
       fecha_inscripcion
INTO   v_ficha
FROM   CLIENTE
WHERE  nro_cliente = v_nro;

-- PASO 4: leemos cada campo con el operador punto.
DBMS_OUTPUT.PUT_LINE('===== FICHA DE CLIENTE =====');
DBMS_OUTPUT.PUT_LINE('RUN          : ' || v_ficha.run);
DBMS_OUTPUT.PUT_LINE('Nombre       : ' || v_ficha.nombre);
DBMS_OUTPUT.PUT_LINE('Direccion    : ' || v_ficha.direccion);
DBMS_OUTPUT.PUT_LINE('Inscrito el  : ' ||
                    TO_CHAR(v_ficha.inscripcion, 'DD/MM/YYYY'));

END;
/
```

Salida esperada

```
===== FICHA DE CLIENTE =====
RUN          : 16044255-7
Nombre       : STEPHANIE DIAZ KOHNERT
Direccion    : Avda. General Velasquez 123 Depto 45
Inscrito el  : 18/10/2010
```

Nota: la fecha de inscripción se genera en el script a partir del año actual, así que el año que veas dependerá de cuándo se cargó la base de datos.

Cuatro cosas que debes retener de este ejercicio

1. TYPE crea el molde; la línea siguiente crea la variable. Son pasos distintos.
2. Un solo SELECT ... INTO carga el RECORD completo.
3. La correspondencia entre columnas y campos es por posición, no por nombre.
4. El operador punto se aplica a la variable, nunca al tipo.

Prueba a romperlo: invierte el orden de direccion y fecha_inscripcion en el SELECT (sin tocar el RECORD) y ejecuta. Anota el número de error que aparece y explica en una línea por qué se produjo. Esta observación te ahorrará horas de depuración más adelante.

Ejercicio 2 — Detalle de un producto de crédito

Dificultad: baja. Tiempo estimado: 8 minutos.

Requerimiento

El área comercial necesita consultar la ficha completa de un producto de crédito del catálogo. Construye un bloque PL/SQL anónimo que, para el crédito de código 2, muestre por pantalla:

- El código y el nombre del crédito.
- La tasa de interés anual.
- El número máximo de cuotas.
- Los primeros 80 caracteres de la descripción.

Restricción

Debes resolverlo con el atributo `%ROWTYPE`, sin definir un tipo `RECORD` propio.

Pistas

- La tabla es `CREDITO` y su clave primaria es `cod_credito`.
- Declara la variable como `CREDITO%ROWTYPE` y cárgala con `SELECT * INTO`.
- Los nombres de los campos del `RECORD` son los mismos nombres de las columnas.
- Para recortar la descripción usa la función `SUBSTR`.

Salida esperada (formato de referencia)

```
===== PRODUCTO DE CREDITO =====
Codigo      : 2
Nombre      : Credito de Consumo
Tasa anual  : 14,5%
Maximo cuotas : 48
Descripcion : Credito para financiar la compra de bienes de consumo o real...
```

Para pensar

Si el requerimiento hubiera pedido solamente el nombre y la tasa, ¿seguirías usando `%ROWTYPE`? Escribe tu respuesta en un comentario al final del archivo.

Ejercicio 3 — Perfil comercial del cliente

Dificultad: media. Este es el ejercicio central de la sesión. Tiempo estimado: 12 minutos.

Requerimiento

El área de riesgo necesita un perfil comercial resumido del cliente para evaluar solicitudes de crédito. Construye un bloque PL/SQL anónimo que, para el cliente número 3, muestre:

Dato	Origen
Número de cliente	CLIENTE.nro_cliente
Nombre completo	CLIENTE (pnombre, snombre, appaterno, apmaterno)
Tipo de cliente	TIPO_CLIENTE.nombre_tipo_cliente
Profesión u oficio	PROFESION_OFICIO.nombre_prof_ofic
Comuna	COMUNA.nombre_comuna
Edad en años cumplidos	Calculado a partir de CLIENTE.fecha_nacimiento
Antigüedad en años	Calculado a partir de CLIENTE.fecha_inscripcion

Restricción

Debes definir un tipo RECORD propio. Antes de escribir el código, responde en un comentario: ¿por qué %ROWTYPE no sirve para este requerimiento?

Pistas

- Necesitas unir cuatro tablas: CLIENTE con TIPO_CLIENTE, con PROFESION_OFICIO y con COMUNA.
- La relación con COMUNA usa tres columnas a la vez: cod_region, cod_provincia y cod_comuna.
- Para la edad y la antigüedad en años: `TRUNC(MONTHS_BETWEEN(SYSDATE, fecha) / 12)`
- Recuerda que el SELECT debe listar las columnas exactamente en el mismo orden en que declaraste los campos del RECORD.

Salida esperada (formato de referencia)

```
===== PERFIL COMERCIAL =====
Cliente N      : 3
Nombre         : STEPHANIE DIAZ KOHNERT
Tipo cliente   : Trabajadores dependientes
Profesion      : Tecnico en computacion
Comuna         : Arica
Edad           : 42 anos
Antigüedad     : 16 anos como cliente
```

Verificación: los valores de tipo de cliente, profesión y comuna dependen de los códigos que tenga el cliente 3 en tu carga de datos. Contrasta tu resultado con una consulta SQL directa antes de dar por bueno el bloque.

Ejercicio 4 — Estado de un crédito con RECORD anidado

Dificultad: alta. Ejercicio de estiramiento. Tiempo estimado: 12 minutos.

Requerimiento

El área de cobranza necesita conocer el estado de avance de un crédito otorgado. Construye un bloque PL/SQL anónimo que, para la solicitud de crédito número 2001, muestre en un solo informe:

Bloque de identificación del crédito:

- Número de solicitud, nombre del producto de crédito, monto del crédito y total de cuotas pactadas.

Bloque de resumen de cuotas (debe ser un RECORD anidado dentro del anterior):

- Cantidad de cuotas ya pagadas.
- Monto total pagado hasta la fecha.
- Monto pendiente, entendido como la suma del valor de las cuotas que aún no registran pago.

Además, calcula y muestra el porcentaje de avance del crédito, con un decimal.

Restricción

El RECORD principal debe contener un segundo RECORD como uno de sus campos.

Pistas

- Define primero el tipo del RECORD interno; solo después podrás usarlo como tipo de un campo del RECORD externo.
- Los datos del crédito salen de CREDITO_CLIENTE unida con CREDITO por cod_credito.
- Los datos de las cuotas salen de CUOTA_CREDITO_CLIENTE, con funciones de grupo.
- Una cuota está pagada cuando fecha_pago_cuota IS NOT NULL. Puedes contarlas con COUNT(fecha_pago_cuota).
- Un SELECT ... INTO **no puede** cargar de una vez un RECORD que contiene otro RECORD. Tendrás que usar dos consultas y cargar los campos indicando la ruta completa, por ejemplo INTO v_estado.cuotas.pagadas.
- Protege las sumas contra el valor NULL con NVL(...).

Salida esperada (formato de referencia)

```
===== ESTADO DEL CREDITO =====
Solicitud N      : 2001
Producto        : Credito Hipotecario
Monto credito   : 2.750.000
Cuotas pactadas : 48
-----
Cuotas pagadas   : 7
Monto pagado     : 401.044
Monto pendiente  : 2.348.972
Avance           : 14,6%
```

Nota: las cifras de cuotas pagadas dependen de la fecha en que se cargó la base de datos, porque el script genera las fechas en función del año actual. Lo que se evalúa es la estructura de tu solución, no que los números coincidan con el ejemplo.

Desafío opcional

Si terminaste los cuatro ejercicios, resuelve este. BANK SOLUTIONS mantiene la tabla CLIENTE_TODOSUMA con los clientes que participan en su programa de puntos.

Construye un bloque PL/SQL anónimo que, para el cliente número 5:

1. Declare una variable de tipo CLIENTE_TODOSUMA%ROWTYPE.
2. Cargue sus campos con el RUN, el nombre completo, el nombre del tipo de cliente y la suma de todos los montos solicitados por ese cliente en CREDITO_CLIENTE.
3. Calcule el monto en pesos del programa como el 1% del total solicitado.
4. Inserte la fila completa con la sintaxis INSERT INTO tabla VALUES variable, sin listar las columnas y sin paréntesis.
5. Confirme la transacción y verifique el resultado con un SELECT.

Autoevaluación

Marca cada afirmación solo si puedes demostrarla con tu propio código.

✓	Puedo...
☐	Explicar con mis palabras la diferencia entre definir un tipo RECORD y declarar una variable de ese tipo.
☐	Declarar un tipo RECORD combinando %TYPE de columnas con tipos de dato explícitos.
☐	Cargar un RECORD completo con un solo SELECT ... INTO y explicar por qué el orden importa.
☐	Justificar cuándo conviene %ROWTYPE y cuándo conviene un RECORD definido por mí.
☐	Construir un RECORD con datos provenientes de más de una tabla y con campos calculados.
☐	Declarar y cargar un RECORD anidado, accediendo a sus campos con puntos encadenados.
☐	Identificar la causa de los errores ORA-06502, PLS-00306 y PLS-00382 cuando aparecen.

Para la próxima clase: un RECORD sirve para agrupar datos distintos de una misma entidad. Pero ¿qué haces si necesitas guardar una lista de valores del mismo tipo, por ejemplo los códigos de los diez productos de inversión del catálogo? Esa es la pregunta que abre la sesión de VARRAY. Revisa la guía antes de llegar.